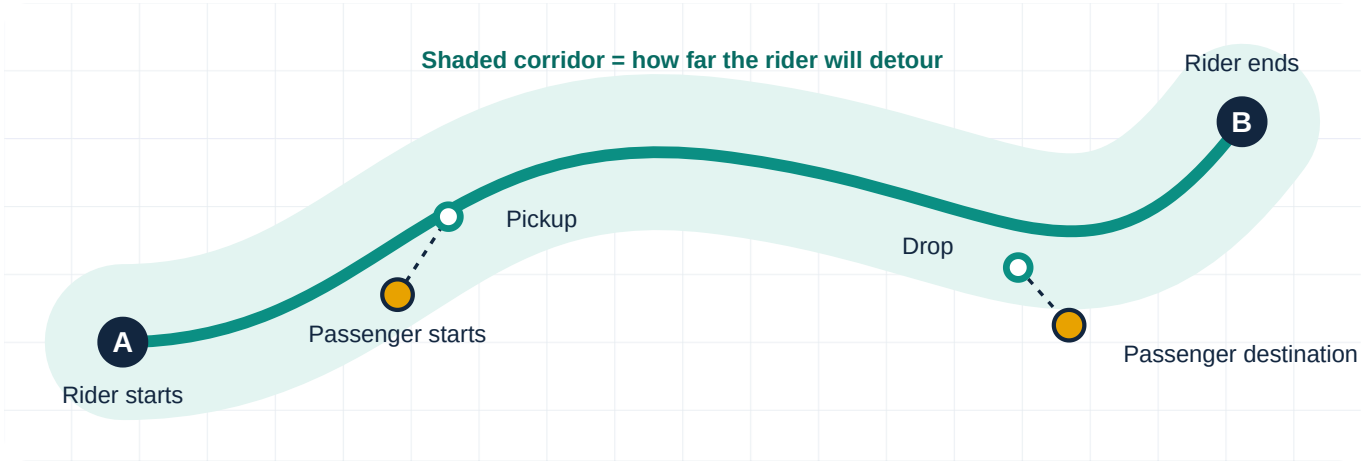
Share the road.
Split the cost.
Nobody profits.

Many people ride alone on a bike or in a car with empty seats, while others pay a lot for a cab or auto on the same road. Our app connects them, and splits only the real cost of the trip.



The idea in one picture: the rider is already going from A to B. A passenger whose start and end both fall inside the corridor can join part of the trip.

What it is

A mobile app where people with a bike or car share spare seats on trips they are already making.

Who it is for

Students, office-goers and daily commuters in Pune.

Money rule

Cost-sharing only. The rider never earns more than the trip really cost. The app takes no commission.

One app, two modes

The same app lets a user offer a ride or find a ride.

Our edge

Smart route matching and AI/ML features that make matches better, safer and cheaper.

Stack

React Native, Node.js, PostgreSQL with PostGIS, and a small Python ML service.

The problem

Roads are full of empty seats, while commuters overpay.

What happens today

- A person rides a bike or drives a car to the same office or college every day, with one or more empty seats.
- Another person going the same way books Rapido, Ola or a cab and pays much more.
- Both lose: one pays too much, the other pays all the fuel alone, and the road gets more crowded.
- Friends of friends sometimes share rides in WhatsApp groups, but it is messy, unsafe to trust, and has no fair way to split money.

What people want

- A cheap ride that is still comfortable.
- To know the other person is verified and safe.
- A fair price that nobody argues about.
- A match that does not make the rider take a long detour.

How this is different from RodBez

RodBez (seen on Shark Tank India) runs one-way taxis and taxi pooling with commercial drivers and earns a commission. Our idea is **peer-to-peer**: ordinary people sharing a trip they are already making, with **no commission and no profit**.

Our solution

A cost-sharing pooling app with smart matching. One app, where each person can be a Rider or a Passenger.

How a trip works

1

The rider posts a trip.

They enter start, destination, time and vehicle, and the app draws the route. The rider can have a bike (1 seat) or a car (up to 3 seats).

2 **A passenger searches.**
They enter where they are and where they want to go. The app finds trips whose route passes close to both points, in the right order, and in the right time window.

3 **The app shows the best matches.**
Each match shows the pickup point, the extra detour, the fair cost share, and the rider's trust score.

4 **The rider accepts.**
Both get a ride code and can see each other's live location, with an SOS button for safety.

5 **They ride and split the cost.**
The app calculates the capped, fair share. The passenger pays the rider directly by UPI, so the money never passes through us.

6 **Both rate each other.**
Ratings update the trust score and also teach the matching model.

Try the fare split

This is the heart of the cost-sharing promise. The cost of the trip is split equally among everyone in the vehicle, **including the rider**. That is why it is cost-sharing, not a taxi fare.

Fare split calculator

Petrol bike | E-bike | Car

Distance (km)	Cost per km (₹)	Tolls and parking (₹)	People in vehicle
12	3.5	0	2 (rider + 1)

Total trip cost	Each person pays	Rider gets back
₹42	₹21	₹21

The cost per km values are placeholders for the demo. The team will calibrate them from real fuel prices and mileage. The app will refuse any price above this calculated cap.

Rules in Maharashtra

We design for cost-sharing so the project stays on the safe side of the law. This is a summary for the team, not legal advice.

The simple rule

A private (white-plate) vehicle may not carry passengers for **hire or reward**. Splitting the real cost of a trip someone is already making is treated very differently from earning money by driving people. That is why we cap every price at the real cost.

Topic	What reports say	What it means for us
Car pooling	Maharashtra's cabinet approved private car pooling in April 2025, through apps registered with the transport department. Reported conditions: 14 pooling trips per week, fares set by the RTA and not above cab rates, mandatory insurance, ID verification, female drivers for women who want them.	Build these limits into the app: weekly trip counter, fare cap, insurance upload, ID check, women-only option.
Bike pooling	Also approved in April 2025, but only for e-bikes and only when the owner is registered on an aggregator's platform.	Add an e-bike filter and badge. Treat petrol-bike pooling as a demo mode.
Bike taxis	Separate from pooling. The state revoked Ola, Uber and Rapido's provisional bike-taxi licences in March 2026, and a temporary framework has been proposed.	Shows enforcement is real. We never call ourselves a taxi service.
Detailed rules	We could not confirm that the detailed pooling rules have been formally notified. Reports on this vary in reliability.	Check with the state transport department before any real launch.
Platform fee	Aggregators need a state licence, and the central 2025 guidelines mention a fee for fresh applications. A percentage commission on private-vehicle trips is risky.	Our project takes no fee at all. Maps are free (OpenStreetMap).

What we say in the report and viva

The platform is a cost-sharing facilitator. It takes no commission, never raises prices, and caps each trip at the actual cost of travel, split between everyone in the vehicle. Real-world operation would need registration with the state transport department.

Features

Tags show priority: MVP must be in the first working version, Next comes after, and Wow is what makes our project stand out.

Core

Sign up with OTP	MVP Phone number login with a one-time code. One account can switch between Rider and Passenger.
Post a trip	MVP Start, end, time, vehicle, free seats, and the route drawn on the map.
Search and match	MVP Finds trips along the passenger's route with the shortest detour and a close time.
Book, accept, ride	MVP Request, accept, ride code at pickup, trip complete.
Fair-cost engine	MVP Calculates the capped, equal split. Shows a clear breakdown to both people.
UPI payment	MVP Passenger pays the rider directly with a UPI link. Simulated in the prototype.
Ratings	MVP Both sides rate each other after the trip.

Trust and safety why people will actually use it

Verified profiles	MVP Phone check first. Then college or company email, then ID and driving licence (mocked in the prototype).
Ride code and masked numbers	MVP Pickup needs a code. Phone numbers stay hidden inside the app.
Live trip sharing and SOS	Next Share the live trip with family. SOS button sends location to chosen contacts.
Helmet check	Next Rider confirms a spare helmet. Passenger confirms wearing it before the trip starts.
Women-only option	Next Women can choose to match only with women riders or passengers.
Insurance and licence check	Next Riders confirm valid documents before posting. Expired ones block new trips.

Creative features the ones that make us different

Corridor matching	Wow Matches by route shape, not by city. A passenger fits if pickup and drop both lie inside the rider's corridor, in the right order.
Partial-route rides	Wow A passenger can ride only the middle part of the trip. The cost is split by distance covered.
Smart pickup points	Wow The app suggests a safe, easy meeting spot near the route instead of a doorstep, which saves the rider time.
Daily commute pools	Wow Regular riders can form a repeating pool with the same people and time every weekday.
Verified circles	Next Private pooling groups for one college, company or housing society.
Savings dashboard	Next Shows money saved, fuel saved and CO_2 avoided, and a friendly campus leaderboard.
E-bike badge	Next Highlights electric rides, which also fits the direction of Maharashtra's rules.
Metro and bus last-mile mode	Next Matches people heading to a station at peak hours.
Weather alert	Next Warns both people about heavy rain before a bike ride is confirmed.

AI and data science

This is where our team's data science knowledge becomes a real part of the product, not decoration.

> **Important: start simple, then add ML**
>
> A new app has no user data. So we build a **rule-based baseline first**, then create realistic **simulated trips** to train and test ML models, and finally compare ML against the baseline. Anything trained on simulated data must be labelled that way in the report. This is honest and still scores well.

The match score

Every possible match gets a score from 0 to 100. In version 1 we pick the weights by hand. In version 2, a model learns them from what people accept and reject.



Example starting weights only. The team will tune them with the simulator.

Where we use AI and ML

Feature	What it does	Technique	Data	Priority
Corridor matching	Finds trips that pass close to the passenger's start and end, in order.	Geospatial queries (PostGIS), route buffers, shortest paths	Road network from OpenStreetMap	MVP
Detour and time windows	Checks the rider's extra distance and the pickup time stay within limits.	Insertion heuristics, shortest-path routing	OSRM routing, trip times	MVP
Learned match ranking	Learns which matches people actually accept and shows those first.	Learning-to-rank (logistic regression, gradient boosting)	Accept and reject history, simulated at first	Wow
Smart pickup points	Finds the best nearby meeting spots from where people usually join.	Clustering (DBSCAN, K-means) on pickup locations	Past pickup and drop points	Wow
Commute pattern detection	Spots people who travel the same way at the same time every week and suggests a pool.	Pattern mining on trip history, time-series grouping	Trip logs	Wow
Demand heatmap and forecast	Shows where and when passengers will look for rides, so riders post at the right time.	Time-series forecasting (Prophet, gradient boosting)	Search and trip counts by area and hour	Next
No-show and cancel prediction	Warns when a match is likely to fall through so we can suggest a backup.	Classification (random forest, XGBoost)	Past cancellations, lead time, rating	Next
Trip-time (ETA) prediction	Better arrival and pickup times than a plain map estimate.	Regression on time of day, distance and weather	Trip duration logs	Next

Feature	What it does	Technique	Data	Priority
Trust score	Combines ratings, verification, completed trips and cancellations into one clear score.	Weighted scoring, later a calibrated model	Ratings and trip outcomes	MVP
Safety anomaly alerts	Flags unusual events: leaving the route, a long stop, or a speed spike.	Rule checks, then anomaly detection (isolation forest)	Live GPS points	Next
Fare fraud check	Catches fake trips or inflated distances meant to earn money.	Anomaly detection and consistency checks	GPS trace vs claimed distance	Next
Review understanding	Reads written reviews and highlights safety complaints for the admin.	Text classification (sentiment and topic)	Review text	Next
Best time to match	Decides whether to match now or wait a few minutes for a better partner.	Reinforcement learning (research idea)	Simulator	Wow stretch

How we build and prove it

1. **Simulator.** A Python script uses Pune's road network from OpenStreetMap to generate thousands of realistic trips between zones, with morning and evening peaks.
2. **Baseline.** Match by nearest start point only. This is the "naive" method we want to beat.
3. **Our matcher.** Corridor matching with detour and time limits, then the learned ranking on top.
4. **Measure.** Compare on match rate, average detour, average waiting time, seats filled and money saved. For classifiers also report accuracy, precision, recall and AUC. For ranking report how often the accepted match is in the top 3.
5. **Pilot.** A small test with 20 to 30 classmates adds real data. Report simulated and real results separately.

▼ **Why a Python ML service next to Node.js?**

Node.js runs the app's API. Python has the best data science libraries (pandas, scikit-learn, XGBoost, OSMnx). So the ML runs in a small separate service (FastAPI) that the Node API calls. This also lets the data science teammates work without touching the mobile app code.

▼ **What about privacy of location data?**

We keep only what is needed, hide phone numbers, store exact live location only during a trip, and ask for consent in the app. India's data protection law (the Digital Personal Data Protection Act, 2023) is the reference to mention in the report.

Learning from others

We looked at real apps and research papers. Here is what we borrow and what we do differently.

Real apps

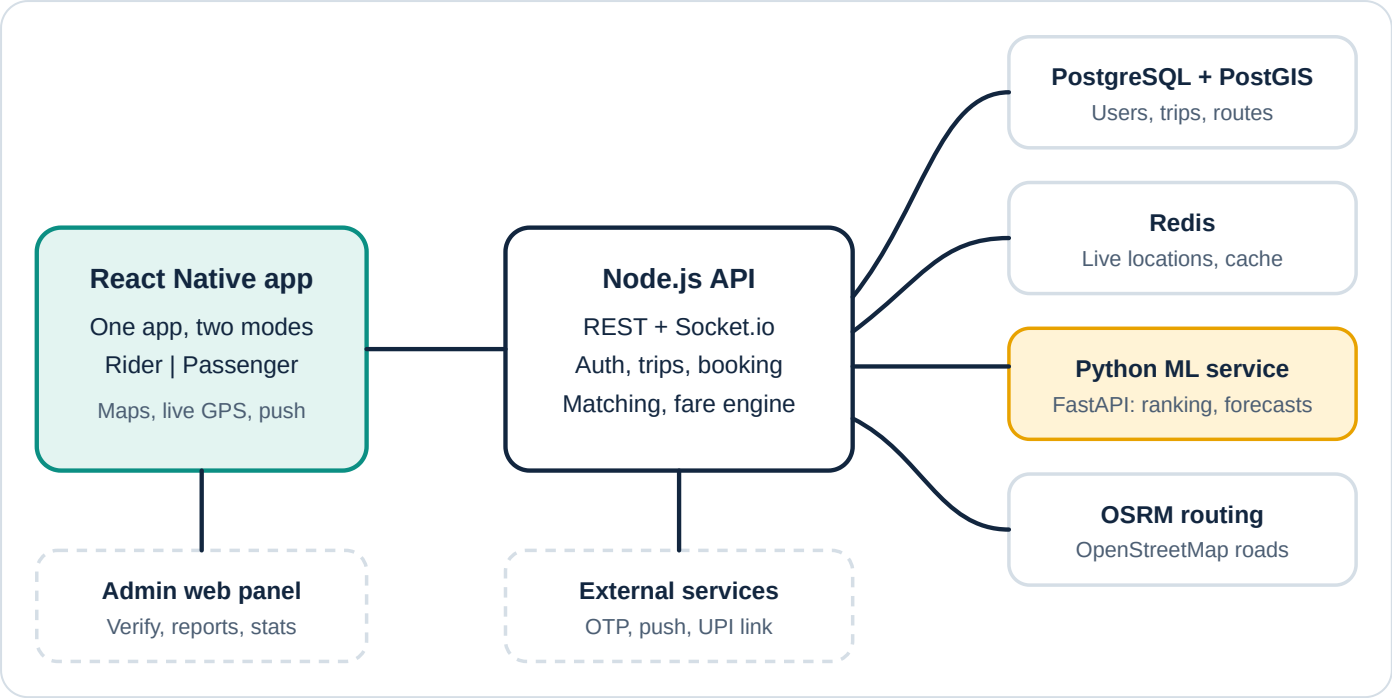
Project	What it does well	What we take	How we differ
Quick Ride (India)	Carpool and bike-pool app. Two-step verification with company email and government ID, filters by gender, route match and verified users, auto-confirm matching, number masking, and a CO_2 tracker.	Layered verification, filters, auto-match, masked numbers, savings meter.	We add a hard cost cap, no profit, learned ranking and corridor matching with partial rides.
BlaBlaCar	Matches drivers with empty seats for intercity trips. Verified profiles, two-way reviews, in-app chat, and copy-recurring-route shortcuts.	Trust through profiles and reviews. Repeat routes.	We focus on daily city commutes, and we do not charge a service fee.
RodBez	One-way taxi and taxi pooling that cuts empty return trips for commercial drivers.	The idea of using empty capacity.	We use private riders, with cost-sharing instead of commission.

Research we can use in our report

- **Dynamic ridesharing with detour limits.** The general problem of matching drivers and riders with time windows is hard to solve exactly, so researchers use fast heuristics. Fast methods check detours using distance estimates first, then exact shortest paths only for promising matches. We copy this two-step idea to keep search fast.
- **Partial ridesharing.** One system lets the passenger reach the driver's route instead of making the driver detour. This is the idea behind our partial-route rides and smart pickup points. That system also reported that a 10 minute matching timeout cut waiting time by around 55%.
- **Personalised ranking.** A research system called GoTogether learns each user's own acceptance habits from which offers they accept or reject. This supports our learned ranking feature.
- **Learning when to match.** A 2025 paper uses reinforcement learning to choose the best time to run matching, which reduced waiting time. This is our stretch idea.

Technology

One React Native app for both riders and passengers, a Node.js backend, and a small Python service for ML.



The app only talks to the Node.js API. The API talks to everything else.

Technology choices

Layer	Choice	Why
Mobile app	React Native with Expo, MapLibre for maps	One codebase for Android and iOS. Expo makes testing on real phones fast.
Backend	Node.js with Express (or NestJS), Socket.io for live updates	Matches the team's preference. Socket.io handles live tracking.
Database	PostgreSQL with PostGIS	PostGIS gives us the route-corridor queries our matching needs.
Cache and live data	Redis	Fast storage for live locations and active trips.
Routing and maps	OpenStreetMap data, OSRM routing server	Free, so there is no map bill and no fee needed for map costs.
ML service	Python, FastAPI, pandas, scikit-learn, XGBoost, OSMnx	Best tools for data science, kept separate from the app.

Layer	Choice	Why
Auth and push	Phone OTP provider and Firebase Cloud Messaging	Standard, with free tiers for a student project.
Payments	UPI deep link (simulated in the prototype)	Money goes rider to passenger, not through us.
Admin panel	Small React web page	For verification, reports and analytics.
Hosting	A cloud VM or free-tier hosting, with Docker	Easy to deploy and to demo.

Technical requirements

Functional

- Phone login and profile with role switching
- Trip posting with route, time, seats and vehicle
- Search and match with corridor, detour and time checks
- Booking flow with accept, ride code, start and end
- Fare engine with cost cap and equal split
- Live location, SOS, ratings, notifications
- Admin: verify users, view reports

Quality targets (proposed)

- Search results in under 2 seconds
- Live location update every few seconds during a trip
- Works on mid-range Android phones and slow networks
- Passwords and tokens stored safely, data sent over HTTPS
- Phone numbers never shown to the other user
- Simple screens in English, with Marathi and Hindi later

What we need from outside

Things the team has to arrange, apart from writing code.

Need	Details	Student-project plan
Map data and routing	OpenStreetMap extract for Pune and a running OSRM server.	Free. Run OSRM in Docker.
OTP and push services	An SMS or phone-auth provider and Firebase project.	Use free tiers or test numbers.

Need	Details	Student-project plan
ID and licence verification	Real Aadhaar or licence checks need approved providers.	Mock the checks. Upload a photo for manual admin review.
Payments	Real UPI collection needs registered payment partners.	Use a UPI deep link or a simulated payment screen.
Test phones	Android phones for each teammate, a few iOS if possible.	Expo Go for quick testing.
Pilot users	20 to 30 volunteers on one college or office corridor.	Ask classmates and friends. Take written consent.
Legal and ethics	Transport rules, data protection, insurance questions.	Cost-sharing design, consent screens, a legal note in the report. Confirm with a guide or faculty.
Hosting	A server for the API, database and ML service.	Free credits or a college server.
Documentation	Report, slides, demo video, GitHub repo.	Keep notes weekly, not at the end.

Plan and roles

A 14-week proposal. Stretch or squeeze it to match our real deadline.

Weeks 1 to 2

Foundation

- Agree scope, name and one pilot corridor in Pune
- Wireframes, database design, repository and team workflow

Weeks 3 to 5

App and backend skeleton

- Login, profiles, role switch, post a trip, map screens
- API, database with PostGIS, OSRM running

Weeks 6 to 8

Matching and booking

- Rule-based corridor matching with detour and time limits
- Booking flow, fare engine with cap, notifications

Weeks 9 to 10

Trust and safety

- Verification flow, ride code, live tracking, SOS, ratings
- Trust score v1

Weeks 11 to 12

AI and data science

- Simulator, learned ranking, pickup-point clustering
- No-show prediction and demand heatmap if time allows

Week 13

Pilot and testing

- Real trial with classmates, bug fixing, feedback survey

Week 14

Results and delivery

- Evaluation chapter, report, slides, demo video, final polish

Suggested team roles

Adjust to the number of people. One person can hold more than one role.

Role	Owns
Mobile lead	React Native screens, maps, live location, push notifications
Backend lead	Node.js API, database, booking, fare engine, deployment
Data and ML lead	Simulator, matching and ranking models, evaluation, Python service
Design, QA and documentation	Wireframes, testing, pilot, report, slides, demo video

How we judge success

- **Matching quality:** match rate, average detour, average wait time and seats filled, compared with the naive baseline.
- **Money:** average saving per passenger compared with a typical cab or bike-taxi fare for the same trip.
- **ML:** top-3 acceptance for ranking, and AUC for the no-show model.
- **Pilot:** number of completed trips, user satisfaction survey, and safety feedback.

Risks and how we handle them

Risk	What we do
Not enough users (a new pooling app is empty at first)	Focus on one corridor or one college. Use the simulator for evaluation and a small pilot for real feedback.
Legal doubt about earning from rides	Cost cap, equal split, no commission, no surge pricing, and a clear legal note in the report.
Safety worries, especially on bikes	Verification, ride code, helmet check, SOS, live sharing, women-only option.
ML results look better than they really are	Always compare against a baseline. Label simulated results as simulated.
GPS drift or poor network	Smooth location points, retry on weak signal, and keep trip data on the phone until it can sync.
Payment complexity	UPI deep link or simulated payments for the prototype.
Too many features	Build the MVP first. Treat Next and Wow items as bonus, in that order.
Privacy of location data	Collect the minimum, mask numbers, get consent, store live location only during trips.

Decisions the team must make

1. **Vehicles:** cars and e-bikes only (legally clearest), or all bikes and cars?
2. **Pilot corridor or group:** one college, one office area, or one route in Pune?
3. **App name** and logo. The current title is a working title.
4. **Which "Wow" features** we commit to. Suggested: corridor matching, partial-route rides, smart pickup points and learned ranking.
5. **Roles and weekly meeting time.**
6. **Final deadline**, so we can fit the 14-week plan.

Glossary

▼ **Words used in this document**

Cost-sharing: splitting the real cost of a trip, with no profit for anyone.

Aggregator: a company or app that runs rides for payment, such as Ola, Uber or Rapido. It needs a state licence.

Corridor: a strip around the rider's route. If both pickup and drop fall inside it, the passenger is a possible match.

Detour: the extra distance or time the rider needs to pick up or drop a passenger.

PostGIS: an add-on for PostgreSQL that understands maps, so we can ask "which routes pass within 500 m of this point?"

Learning-to-rank: an ML method that learns the best order for a list, here the order of match suggestions.

Baseline: a simple method we must beat, so we can prove our smarter method really helps.

Cold start: the problem that a new app has no data and no users yet.

Simulator: a program that creates realistic fake trips so we can test and train without real users.

MVP: the smallest working version of the app that still solves the core problem.

Sources

For the team to read more. Legal items change often, so verify before relying on them.

- Research: "Timing the Match: A Deep Reinforcement Learning Approach for Ride-Hailing and Ride-Pooling Services" (arXiv 2503.13200)
- Research: "Fast, Exact and Scalable Dynamic Ridesharing" (arXiv 2011.02601)
- Research: "A Machine-Learned Ranking Algorithm for Dynamic and Personalised Car Pooling Services" (GoTogether, arXiv 2307.05697)
- Research: "Dynamic Carpooling in Urban Areas: Design and Experimentation with a Multi-Objective Route Matching Algorithm" (Sustainability, 2017)
- Research: "A Dynamic Tree Algorithm for Peer-to-Peer Ride-sharing Matching" (arXiv 2105.13078)
- Product: Quick Ride's blog posts on profile verification, safety features and top features
- Product: BlaBlaCar overviews on how it works, trust and verification
- Legal context: Business Standard report on app-based carpooling (July 2026), reports on Maharashtra cabinet approval of bike and car pooling (April 2025), and reports on Maharashtra's bike-taxi licence actions (March

2026)

Working document for the project team. Legal points are a summary of public reports, not legal advice. Numbers in examples are placeholders to be calibrated.